ECEN 723 Final Project - Phase C

V-group Verification Report on the i-group Signal System

Team 10 - V-group (Dhruv Nandwani, Brian Hsieh)

Spring 2026

1. Introduction

In Phase C of the final project, our v-group is paired with the i-group, and our task is to formally verify that the i-group's signal-system component satisfies the three mandatory v-group properties from the project specification, plus one additional property of our own choosing. We use Spin (the same model checker we introduced in Phase B Task B.2) so that the workflow, tooling, and reporting style carry over directly from our prior submission.

The component under verification is the i-group's per-intersection signal controller, which is implemented in i_group/scheduler.py and i_group/policy.py and runs once per simulator step on each of the nine intersections in the 3x3 grid defined by shared/topology.py. The controller maintains the active green direction for an intersection, ticks a per-direction green-time counter, and decides at each step whether to keep the current phase, voluntarily switch to a busier direction, or force-switch when MAX_GREEN is reached. The properties we verify are the spec-mandated three (every direction is eventually green, every direction is infinitely often green, exactly one direction is green at any time) and one we chose ourselves: that the active green phase is always one of the topology-valid directions for the intersection.

The remainder of this report describes the tool we used and how we set it up, the modeling abstractions we made, each of the four properties together with its Promela encoding and Spin result, the one counterexample we found and the proposed fix, and the limitations of our verification.

2. Tool Selection

We selected Spin again for Phase C because the modeling and property language we need are the same ones we already exercised in Phase B Task B.2. The signal controller is a reactive process whose decisions depend only on its local phase, timer, and direction-count inputs; this fits cleanly into a single Promela proctype and lets us state the relevant properties in linear temporal logic (LTL). Spin's verifier explores all reachable states systematically and, for safety properties, gives a complete answer over the modeled state space; for liveness properties, the -f flag enables weak fairness, which matches the simulator's once-per-step controller invocation.

We considered using assertions only, as we did in Phase B, but two of the three mandatory v-group properties (every direction is eventually green; every direction is infinitely often green) are liveness statements and are not naturally expressible as assertions. Spin's LTL support lets us state these directly using the [] (always) and <> (eventually) operators, so we kept the same tool but moved from assertions to ltl claims for this phase.

3. Environment Setup

All verification runs were carried out on a macOS machine with Spin and gcc installed via Homebrew. The exact versions captured by run_all.sh are recorded in logs/00_spin_version.txt and reproduced in Appendix A; in our environment Spin reports Version 6.5.2 and gcc reports Apple clang 15.0.0. The three

Promela source files (signal_check.pml, signal_check_corner.pml, signal_check_corner_fixed.pml) and the driver script (run_all.sh) are placed in the same directory; running the driver creates a logs/ subdirectory containing one text file per verification.

4. Basic Spin Workflow

The workflow we follow for each property is identical to the five-step process we documented in Phase B, with one addition for liveness. The steps are:

1. Model the controller in Promela (the .pml files in this submission).
2. State the property as an ltl claim inside the same .pml file.
3. Invoke Spin in three explicit steps: spin -a -DSEL_<name> <file> to generate the pan.c verifier, gcc -O2 -o pan pan.c to compile it, and ./pan to run it. The -DSEL_<name> preprocessor flag activates exactly one #ifdef-wrapped LTL claim in the .pml source; this is portable across Spin versions and avoids the broken -ltl <name> flag in macOS-Homebrew Spin (which silently treats the property name as the input filename).
4. For liveness properties (P1, P2), pass -a -f to ./pan (not to spin) to enable weak fairness so the controller is not allowed to stutter forever.
5. Read the verifier's summary line; errors: 0 means the property holds for every reachable state, errors: 1 means Spin found a counterexample, which can then be replayed with spin -t <file>.

The driver script run_all.sh runs every property in this report in sequence and saves the full verifier output for each one under logs/. Section 6 onward refers to those individual logs by name.

5. System Under Verification and Modeling Strategy

5.1 The i-group Signal System

The i-group's signal system maintains, for each intersection, a phase (one of N, S, E, W), a green-time counter timer, and a round-robin pointer cycle_index that is advanced each time a phase change occurs. On every simulator step, policy.SignalPolicy.decide() is called for each intersection with the current per-direction approaching-vehicle counts and returns either the current phase (a tick) or a new direction (a switch). The decision logic, in priority order from policy.py, is:

6. If the current phase is not in the topology-valid set for this intersection, switch to the cycle-priority-best valid direction (topology-invalid correction).
7. If timer >= MAX_GREEN, force-switch to the cycle-priority-best other valid direction.
8. Otherwise, if timer >= MIN_GREEN and some other direction has strictly more approaching cars than the current direction, voluntarily switch to that direction (using cycle_priority as a tiebreaker).
9. Otherwise, keep the current phase and increment timer.

MIN_GREEN and MAX_GREEN are 10 and 40 simulator steps respectively, taken from i_group/scheduler.py. cycle_priority(d) is defined as the negation of (d - cycle_index + 4) % 4, so ties go to the direction nearest to the round-robin pointer. The simulator's published throughput numbers for this controller are 33 successful tours per hour at 6 cars and a best of 48 at 12 cars (simulator/logs/), establishing that the controller is functional under realistic load; our task is to give a stronger temporal-logic guarantee about its behavior.

5.2 Modeling Strategy and Abstractions

Three abstractions make the system tractable for Spin while still being faithful to the parts of the design our four properties depend on.

Number of intersections. The grid contains nine intersections, but each intersection's policy state (phase, timer, cycle_index) is independent of every other intersection's; nothing in policy.py or scheduler.py reads from a neighbor. Modeling all nine in parallel would inflate the state space by roughly nine orders of magnitude with no new behavior, so we model one worst-case intersection in detail and lift the result to the grid by per-intersection independence. The worst case for properties P1, P2, P3 is the centre intersection I11, where all four directions are topology-valid (every direction can carry green); the worst case for the topology-validity property P4 is a corner intersection (we use I00, where only S and E are valid).

Vehicle demand. The voluntary-switch branch of policy.py is the only one that reads the per-direction count; the other three branches depend only on the current phase and timer. To isolate the controller's round-robin tiebreaker (which is the mechanism that prevents starvation under our liveness properties), we model uniform demand: every direction has count 1 at every step. Under uniform demand, the strict-greater test best_other_count > current_count never holds, so voluntary switches do not fire, and phase changes are driven entirely by MAX_GREEN with cycle_priority breaking ties. This is the right abstraction for liveness because it removes the input-controlled adversary that could otherwise starve a direction by always raising the count of some other direction at force-switch moments; in particular, P1 and P2 fail under arbitrary demand and are intended to characterise the round-robin guarantee rather than a guarantee against arbitrary inputs.

Time. Each iteration of the controller's outer loop corresponds to one simulator step. We do not model wall-clock time, sensor delays, or vehicle dynamics; the timer counter is the only notion of time the controller itself uses.

6. Example 1: P1 - Every Direction Eventually Green

6.1 Model Description

The model is signal_check.pml, instantiated at I11 (all four directions valid). The controller proctype mirrors the four-branch decision in policy.py with the voluntary branch suppressed by uniform demand, and with the topology-invalid correction branch unreachable at I11. Phase, timer, cycle_index, and four mutually-exclusive booleans green_N..green_W are global; the four booleans are kept synchronised with phase by the switch_phase_to inline so that property P3 can be expressed structurally.

6.2 Property

For each direction d in {N, S, E, W}, eventually phase equals d. In Promela:

```
ltl p1_eventually_N { <> (phase == N) }
ltl p1_eventually_S { <> (phase == S) }
ltl p1_eventually_E { <> (phase == E) }
ltl p1_eventually_W { <> (phase == W) }
```

6.3 Procedure

```
spin -a -DSEL_p1_N signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p1_S signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p1_E signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p1_W signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
```

The -f flag enables weak fairness, which is required because without it Spin would consider an execution in which the controller stutters forever from the initial state and never executes its outer loop; under the simulator's actual semantics that loop runs once per step, which weak fairness captures.

6.4 Result

All four runs produced:

- errors: 0
- 0 acceptance cycles found in the never-claim product

Interpretation: in every fair infinite execution of the controller, every direction is reached. Combined with per-intersection independence, this lifts to the spec property at every intersection in the grid. Depth reached: N at depth 0 (the initial state already satisfies phase == N), S at 743, E at 1441, W at 2139, reflecting the round-robin order in which the controller visits directions before completing a full cycle of MAX_GREEN ticks. Logs: p1_eventually_N.txt through p1_eventually_W.txt.

7. Example 2: P2 - Every Direction Infinitely Often Green

7.1 Model Description

Identical to Section 6.1 (signal_check.pml, I11, all four directions valid). P2 is strictly stronger than P1, so it shares the model.

7.2 Property

For each direction d, infinitely often phase equals d. In Promela:

```
ltl p2_inf_often_N { [] <> (phase == N) }
ltl p2_inf_often_S { [] <> (phase == S) }
ltl p2_inf_often_E { [] <> (phase == E) }
ltl p2_inf_often_W { [] <> (phase == W) }
```

This is the unconditional fairness property: no direction is starved, even in infinite runs.

7.3 Procedure

```
spin -a -DSEL_p2_N signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p2_S signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p2_E signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p2_W signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
```

7.4 Result

All four runs produced:

- errors: 0
- 0 acceptance cycles found

Interpretation: Spin found no fair execution that visits any direction only finitely often. Under our uniform-demand abstraction, the controller cycles through N -> S -> E -> W -> N every 4 * MAX_GREEN = 160 steps, so each direction is reached on a fixed period and starvation is impossible. All four runs reached depth 3633, exhausting the full reachable state space. Logs: p2_inf_often_N.txt through p2_inf_often_W.txt.

8. Example 3: P3 - Exactly One Direction Green at Any Time

8.1 Model Description

Same Promela file (signal_check.pml). The four green_* booleans are written in lockstep by switch_phase_to, so their joint state is determined by phase. We chose a structural (boolean-per-direction) encoding rather than a derived property like phase in {N,S,E,W} because the boolean encoding catches an entire class of latent encoding bugs: any future code path that updated phase without updating one of the booleans (or vice versa) would falsify the property and Spin would report a counterexample. The derived form would silently agree with itself.

8.2 Property

Always, exactly one of the four green booleans is true.

```
ltl p3_one_green {
    [] ( ( green_N && !green_S && !green_E && !green_W) ||
         (!green_N &&  green_S && !green_E && !green_W) ||
         (!green_N && !green_S &&  green_E && !green_W) ||
         (!green_N && !green_S && !green_E &&  green_W) )
}
```

8.3 Procedure

```
spin -a -DSEL_p3_one_green signal_check.pml
gcc -O2 -o pan pan.c
./pan
```

P3 is a safety property and does not require -f; the verifier explores all reachable states and confirms the invariant holds at each one.

8.4 Result

The run produced:

- errors: 0
- State space fully explored under default Spin search

Interpretation: in every reachable state of the controller, exactly one green_* boolean is true. Because the greens are written atomically with phase, this means there is no instant at which two directions show green simultaneously. The verifier reached depth 3633, exhausting the full reachable state space. Note that P3 must hold at depth 0, before any process has stepped, so the model declares the four green_* booleans with explicit inline initialisers (green_N = 1, others = 0) matching phase = N; relying on Promela's auto-zero default would falsely violate P3 at the initial state. Log: p3_one_green.txt.

9. Example 4: P4 - Topology Validity at the Corner Intersection (Self-chosen)

9.1 Why this property

The three mandatory properties characterise scheduling fairness and mutual exclusion at a worst-case interior intersection. The structurally weakest intersections are the four corners of the 3x3 grid, where two of the four cardinal directions have no outgoing road in shared/topology.py. Granting green to one of those non-existent legs is meaningless from a traffic-flow perspective and is forbidden by the topology specification. We chose this as our v-group property because, unlike a hard-coded constant such as MAX_GREEN, it expresses an interaction between two independently maintained pieces of the system (the per-intersection valid set in topology.py and the per-intersection initial phase in scheduler.py); a discrepancy between them is exactly the kind of bug that simulation may not exercise but that model checking can find.

9.2 Model Description

We instantiate this property at I00 (top-left corner), where the topology-valid set is {S, E}. Two model files share the same controller logic but differ in their inline initialisers: signal_check_corner.pml declares byte phase = N (the original scheduler's default), and signal_check_corner_fixed.pml declares byte phase = E (the proposed fix). They differ from signal_check.pml in two ways: an IS_VALID(d) macro encodes the corner's valid set, and the Promela controller includes the topology-invalid correction branch from policy.py (decision branch 0 in our enumeration of Section 5.1), which is unreachable at I11 but critical here. The initial phase is set inline at the top of each file, ensuring Spin sees the correct (or buggy) value at depth 0; relying on Promela's auto-zero default or setting it inside the controller's first atomic block would be too late, because LTL formulas are evaluated at the initial state before any process has stepped.

9.3 Property

Always, phase is one of I00's valid directions.

```
ltl p4_topology_valid { [] (phase == S || phase == E) }
```

9.4 Procedure (original scheduler)

The original i_group/scheduler.py uses initialize(self, intersection_ids, default_phase="N"), so signal_check_corner.pml declares byte phase = N inline at the top of the file, exactly mirroring the simulator's start-up state. Since this file contains a single LTL claim, no -DSEL_<name> flag is required - Spin auto-selects the only claim as the never claim.

```
spin -a signal_check_corner.pml
gcc -O2 -o pan pan.c
./pan
spin -t signal_check_corner.pml
```

9.5 Result (original scheduler)

The verifier produced:

- errors: 1
- safety violation reported at depth 0 (the initial state itself)
- pan: ltl formula p4_topology_valid violated

Replaying the trail with spin -t signal_check_corner.pml prints the initial state, in which phase = 0 (i.e. N). N is not in I00's valid set {S, E}, so the property is violated by the very first state of every execution. In the simulator this transient is corrected on the first call to policy.SignalPolicy.decide(), which detects the invalid phase via its topology-invalid-correction branch and switches to a valid direction immediately; no vehicle is ever queried about the phase before that first decide. Spin nevertheless flags the literal property violation at the initial state, which is the correct behaviour for a safety LTL formula. Logs: p4_topology_valid_original.txt and p4_counterexample_trail.txt.

9.6 Proposed Fix and Re-verification

The fix is a one-line change to scheduler.initialize that takes a per-intersection valid-phase callback rather than a single global default:

```
- def initialize(self, intersection_ids, default_phase="N"):
-     for iid in intersection_ids:
-         self._phase[iid] = default_phase
+ def initialize(self, intersection_ids, valid_phases_for):
+     for iid in intersection_ids:
+         self._phase[iid] = next(iter(valid_phases_for(iid)))
```

where valid_phases_for(iid) is supplied by the caller from shared/topology.py. signal_check_corner_fixed.pml encodes this fix by declaring byte phase = E inline (an element of I00's valid set {S, E}). Re-running Spin against the fixed model:

```
spin -a signal_check_corner_fixed.pml
gcc -O2 -o pan pan.c
./pan
```

produces:

- errors: 0

Interpretation: with the proposed initialisation, the controller starts in a topology-valid phase and never leaves the valid set thereafter (the Step A scan in the controller only proposes IS_VALID(d) directions as switch targets). The verifier reached depth 1990, fully exploring the reachable state space at the corner. The bug is real but its observable impact is limited to the initial state; the correction branch in policy.py masks it from the simulator. Log: p4_topology_valid_fixed.txt.

10. Discussion

The verification confirms the three mandatory v-group properties on the worst-case intersection under uniform demand and on the round-robin tiebreaker that drives switching when the voluntary branch is dormant. Together with the per-intersection independence of policy.py and scheduler.py, the result lifts to the full 3x3 grid for every intersection in the topology, and the boolean-per-direction encoding of P3 makes the safety result robust against future encoding-only refactors. P4 surfaced a genuine but observable-only-at-initialisation bug in scheduler.initialize and we proposed a one-line fix; signal_check_corner_fixed.pml verifies that the fix closes the gap.

The most important limitation is the uniform-demand abstraction. Under arbitrary demand, the voluntary-switch branch can fire at any timer value above MIN_GREEN, and an adversarial input sequence can starve a particular direction (consider an input that holds cnt[N] = 0 forever while keeping cnt[S] high). P1 and P2 are not intended to hold against such adversaries; they characterise the round-robin guarantee, which is what the controller's force-switch and cycle_priority logic deliver. Verifying liveness under arbitrary demand would require either a stronger fairness assumption on the inputs or a property that explicitly conditions on demand, and is out of scope for this report.

Other limitations are smaller: we do not model sensor delays, partial observability, or the inter-intersection coordination that would be needed to verify upstream-downstream queue interactions; we do not model the actual vehicle traffic generated by the i-group simulator; and we model exactly one of the nine intersections in detail (the lifting argument relies on policy.py's per-intersection independence, which we verified by inspection).

11. Simulation Throughput and Safety Metrics

The Phase C specification also requires reporting simulation throughput and safety counters. The baseline run uses a 6-car fleet simulated for 3600 seconds (1 simulated hour, 1800 steps). A bottleneck sweep scales the fleet from 6 to 60 cars to find the peak-throughput configuration.

11.1 Baseline Run (6 Cars, 1 Hour)

Metric	Value
Simulated duration	1 hour (3600 s, 1800 steps)
Fleet size	6 cars
Completed tours	33

Throughput	33.0 tours / hr
Collisions	0
Red-light violations	0
Illegal running directions	0
U-turns	0

11.2 Bottleneck Search (6 – 60 Cars)

A fleet-size sweep from 6 to 60 cars was run, each for one simulated hour. Throughput peaked at 12 cars (48.0 tours/hr) and first dropped at 14 cars.

Cars	Throughput (tours/hr)	Collisions	Red-light	Illegal Dir	U-turns	Status
6	33.0	0	0	0	0	ok
8	42.0	0	0	0	0	ok
10	46.0	0	0	0	0	ok
12	48.0	0	0	0	0	ok ← peak
14	47.0	0	0	0	0	DROP ← bottleneck
16	46.0	0	0	0	0	DROP
18	40.0	0	0	0	0	DROP
20	43.0	0	0	0	0	ok
22	43.0	0	0	0	0	ok
24	39.0	0	0	0	0	DROP
60	28.0	0	0	0	0	DROP

All fleet sizes produced zero collisions, red-light violations, illegal running directions, and U-turns, consistent with the formal verification results: P3 safety (exactly one green at any time) and P1/P2 liveness (every direction eventually and infinitely often green) hold at every intersection.

12. Conclusion

We modelled the i-group signal controller in Promela and used Spin to verify the three v-group spec properties (every direction eventually green, every direction infinitely often green, exactly one direction green at any time) on a worst-case interior intersection under uniform demand, and one self-chosen property (topology validity) on a corner intersection. P1, P2, and P3 hold; P4 fails on the original scheduler with a one-step counterexample at initialisation, and holds after a one-line patch to scheduler.initialize that we propose. The Promela files (signal_check.pml, signal_check_corner.pml, signal_check_corner_fixed.pml), the driver script (run_all.sh), and the saved logs are submitted alongside this report.

Appendix A. Verification Commands and Logs

Environment:

```
spin -V
gcc --version | head -1
```

P3 - safety:

```
spin -a -DSEL_p3_one_green signal_check.pml
gcc -O2 -o pan pan.c
./pan
```

P1 - liveness, every direction eventually green:

```
spin -a -DSEL_p1_N signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p1_S signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p1_E signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p1_W signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
```

P2 - liveness, infinitely often green:

```
spin -a -DSEL_p2_N signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p2_S signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p2_E signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
spin -a -DSEL_p2_W signal_check.pml
gcc -O2 -o pan pan.c
./pan -a -f
```

P4 - topology validity (original, expected counterexample):

```
spin -a signal_check_corner.pml
gcc -O2 -o pan pan.c
./pan
spin -t signal_check_corner.pml
```

P4 - topology validity (fixed, expected pass):

```
spin -a signal_check_corner_fixed.pml
gcc -O2 -o pan pan.c
./pan
```

All commands above are executed in sequence by run_all.sh, which writes their output to logs/.

Appendix B. Screenshot Inventory

Seven screenshots are included with this submission to provide visual evidence of each Spin run. They are named S0 through S6 to match the table below.

- S0 - spin -V banner and gcc version.

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -V
Spin Version 6.5.2 -- 6 December 2019
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC %
```

-
- S1 - P3 errors: 0 (signal_check.pml, p3_one_green).

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -a -DSEL_p3_one_green signal_check.pml && gcc -O2 -o pan pan.c && ./pan
ltl p3_one_green: [] (((((((green_N) && (! (green_S))) && (! (green_E))) && (! (green_W))) || ((((! (green_N)) && (green_S)) && (! (green_E)
))) && (green_W)))
warning: never claim + accept labels requires -a flag to fully verify

(Spin Version 6.5.2 -- 6 December 2019)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p3_one_green)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     - (not selected)
        invalid end states      - (disabled by never claim)

State-vector 36 byte, depth reached 3633, errors: 0
      208 states, stored
        1 states, matched
      209 transitions (= stored+matched)
     3218 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.013       equivalent memory usage for states (stored*(State-vector + overhead))
    0.286       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage


unreached in proctype Controller
        signal_check.pml:137, state 46, "-end-"
        (1 of 46 states)
unreached in init
        (0 of 2 states)
unreached in claim p3_one_green
        _spin_nvr.tmp:8, state 10, "-end-"
        (1 of 10 states)
```

- pan: elapsed time 0 seconds
- S2 - P1 errors: 0 (signal_check.pml, p1_eventually_W shown as representative).

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -a -DSEL_p2_W signal_check.pml && gcc -O2 -o pan pan.c && ./pan -a -f
ltl p2_inf_often_W: [] (<> ((phase==3)))

(Spin Version 6.5.2 -- 6 December 2019)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p2_inf_often_W)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness enabled)
        invalid end states      - (disabled by never claim)

State-vector 36 byte, depth reached 3633, errors: 0
      374 states, stored (538 visited)
      329 states, matched
      867 transitions (= visited+matched)
    10853 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.023       equivalent memory usage for states (stored*(State-vector + overhead))
    0.287       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage


unreached in proctype Controller
        signal_check.pml:137, state 46, "-end-"
        (1 of 46 states)
unreached in init
        (0 of 2 states)
unreached in claim p2_inf_often_W
        _spin_nvr.tmp:10, state 13, "-end-"
        (1 of 13 states)

pan: elapsed time 0 seconds
```

- S3 - P2 errors: 0 (signal_check.pml, p2_inf_often_W shown as representative).

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -a -DSEL_p2_W signal_check.pml && gcc -O2 -o pan pan.c && ./pan -a -f
ltl p2_inf_often_W: [] (<> ((phase==3)))

(Spin Version 6.5.2 -- 6 December 2019)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p2_inf_often_W)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness enabled)
        invalid end states      - (disabled by never claim)

State-vector 36 byte, depth reached 3633, errors: 0
      374 states, stored (538 visited)
      329 states, matched
      867 transitions (= visited+matched)
    10853 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.023       equivalent memory usage for states (stored*(State-vector + overhead))
    0.287       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage


unreached in proctype Controller
        signal_check.pml:137, state 46, "-end-"
        (1 of 46 states)
unreached in init
        (0 of 2 states)
unreached in claim p2_inf_often_W
        _spin_nvr.tmp:10, state 13, "-end-"
        (1 of 13 states)

pan: elapsed time 0 seconds
```

- S4 - P4 errors: 1 with violation message (signal_check_corner.pml).

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -a signal_check_corner.pml && gcc -O2 -o pan pan.c && ./pan
ltl p4_topology_valid: [] (((phase==1)) || ((phase==2)))
warning: never claim + accept labels requires -a flag to fully verify
pan:1: assertion violated  !( !(((phase==1)||(phase==2)))) (at depth 0)
pan: wrote signal_check_corner.pml.trail

(Spin Version 6.5.2 -- 6 December 2019)
Warning: Search not completed
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p4_topology_valid)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     - (not selected)
        invalid end states      - (disabled by never claim)

State-vector 36 byte, depth reached 0, errors: 1
        1 states, stored
        0 states, matched
        1 transitions (= stored+matched)
        0 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.000       equivalent memory usage for states (stored*(State-vector + overhead))
    0.286       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage
```

- `pan: elapsed time 0 seconds`
- S5 - P4 trail replay showing phase = N at the initial state (spin -t).

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -t signal_check_corner.pml
ltl p4_topology_valid: [] (((phase==1)) || ((phase==2)))
spin: _spin_nvr.tmp:3, Error: assertion violated
spin: text of failed assertion: assert(!(!(((phase==1)||(phase==2)))))
Never claim moves to line 3     [assert(!(!(((phase==1)||(phase==2)))))]
spin: trail ends after 1 steps
#processes: 2
                phase = 0
                timer = 0
                cycle_index = 0
                green_N = 1
                green_S = 0
                green_E = 0
                green_W = 0
  1:    proc  1 (:init::1) signal_check_corner.pml:128 (state 1)
  1:    proc  0 (Controller:1) signal_check_corner.pml:88 (state 52)
  1:    proc  - (p4_topology_valid:1) _spin_nvr.tmp:2 (state 6)
2 processes created
```

- S6 - P4 errors: 0 after fix (signal_check_corner_fixed.pml).

```
(base) dhruvnandwani@Dhruvs-MacBook-Air-4 phaseC % spin -a signal_check_corner_fixed.pml && gcc -O2 -o pan pan.c && ./pan
ltl p4_topology_valid: [] (((phase==1)) || ((phase==2)))
warning: never claim + accept labels requires -a flag to fully verify

(Spin Version 6.5.2 -- 6 December 2019)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p4_topology_valid)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     - (not selected)
        invalid end states      - (disabled by never claim)

State-vector 36 byte, depth reached 1990, errors: 0
      126 states, stored
        1 states, matched
      127 transitions (= stored+matched)
     1739 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.008       equivalent memory usage for states (stored*(State-vector + overhead))
    0.286       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage


unreached in proctype Controller
        signal_check_corner_fixed.pml:112, state 16, "best_dist = dist"
        signal_check_corner_fixed.pml:113, state 18, "(1)"
        signal_check_corner_fixed.pml:74, state 32, "cycle_index = ((best+1)%4)"
        signal_check_corner_fixed.pml:75, state 33, "green_N = (best==0)"
        signal_check_corner_fixed.pml:76, state 34, "green_S = (best==1)"
        signal_check_corner_fixed.pml:77, state 35, "green_E = (best==2)"
        signal_check_corner_fixed.pml:78, state 36, "green_W = (best==3)"
        signal_check_corner_fixed.pml:130, state 55, "-end-"
        (8 of 55 states)
unreached in init
        (0 of 2 states)
unreached in claim p4_topology_valid
        _spin_nvr.tmp:8, state 10, "-end-"
        (1 of 10 states)

pan: elapsed time 0 seconds
```

All other Spin output is captured as text under logs/ and is available in full in the submission archive.